

Developer Summary: Real $\mathbf{Spin}(4,4)$, Split Octonions, Triality, and Einstein-Spinor Dynamics

A repository-wide engineering, mathematical, numerical, provenance, and release guide

Reproducible exact-real implementation

September 2026

Contents

0.1	Purpose	4
1	Repository contract	4
1.1	Supported checkout	4
1.2	License and implementation policy	4
1.3	Determinism policy	4
1.4	Private and external inputs	5
1.5	Tested toolchain and required capabilities	5
1.6	Clone and bootstrap	6
1.7	Configuration ownership	6
1.8	Audit disposition vocabulary	6
2	Top-level architecture	7
2.1	Developer path index	7
3	Build and verification graph	8
3.1	Generator, artifact, checker, and gate ownership	8
3.2	Generated Rust contract	10
3.3	Notebook generation and execution	10
3.4	Logged execution and backup behavior	11
4	Phase map	11

4.1	Phase 0: source audit and bootstrap	11
4.2	Phase 1: exact algebra and triality	12
4.3	Phase 2: triality transport	12
4.4	Phase 3: homogeneous spinor cosmology	12
4.5	Phase 4: notebooks and publications	12
4.6	Phase 5: curved Levi-Civita Einstein-spinor model	12
4.7	Phase 6: Weitzenboeck spin geometry	13
4.8	Phase 7: $\{x_0, \dots, x_7\}$ component refinement	13
4.9	Phase checkpoint matrix	13
4.10	Release lineage	15
4.11	Canonical artifact snapshot	15
5	Rust application architecture	16
5.1	Common numerical artifact schema	16
5.2	Error semantics	17
6	Exact mathematical core	17
6.1	Clifford representation	17
6.2	Split octonions	17
6.3	Triality	18
7	Curved spin geometry	18
8	Einstein-spinor numerical method	18
9	Verification strategy	19
9.1	Layered independence	19
9.2	Phase gates	19
9.3	Publication checks	19
10	Developer workflows	20
10.1	Fast repository status	20
10.2	Complete Python regression suite	20
10.3	Full current verification sequence	20

10.4 Rust quality checks	20
10.5 Publication regeneration	21
10.6 Common failure modes	21
10.7 Changing the solver pin	22
10.8 Adding a new source root or scientific artifact	22
11 Canonical artifacts and provenance	22
11.1 Current publication snapshot	23
11.2 Authority hierarchy	24
12 Release discipline	24
13 Known boundaries and non-claims	25
14 Change-impact guide	25
15 Developer checklist	26
16 Documentation map	26
17 Rebuilding this summary	27

0.1 Purpose

This is the developer summary for the complete `dirac` repository. It is not an executive summary. It explains what the project implements, how its parts fit together, which artifacts are authoritative, how every phase is regenerated, what is independently checked, and where the scientific claims stop.

The repository combines five concerns that must remain distinct:

1. exact real algebra for $C1(4,4)$, half-spin representations, split octonions, and split-real triality;
2. reproducible numerical applications built on a pinned safe-Rust port of SUNDIALS 7.8.0 CVODE;
3. curved Levi-Civita and Weitzenboeck spin geometry in signature $(4,4)$;
4. a commuting classical 16-component Einstein-spinor dark-sector model; and
5. byte-level source audit, notebook execution, publication, and release verification.

The primary developer rule is that generated output is not accepted merely because it can be regenerated. Exact claims, numerical claims, document semantics, hashes, and cross-clone behavior are checked through independent paths.

1 Repository contract

1.1 Supported checkout

The canonical development checkout is `C:\Users\nsh\Developer\code\vscode\dirac` on Windows. PowerShell is the primary shell. Bash/WSL gates resolve the same Windows `python.exe`, `cargo.exe`, `wolframscript.exe`, and MiKTeX `pdflatex.exe` where required so cross-shell verification exercises the same numerical and publication tools.

1.2 License and implementation policy

The root Rust workspace is GPL-3.0-or-later. The solver is a read-only Git submodule under `vendor/sundials_rs`, pinned to commit `d1836e6a279d63a90fe2839a0020123245487e76`. Application behavior belongs in root-owned crates under `studies/`; the vendored solver is not modified to add project-specific physics.

1.3 Determinism policy

Canonical JSON, CSV, generated Rust, notebooks, LaTeX, PDFs, and audit files are hash checked. `.gitattributes` pins line endings for hash-sensitive text and marks PDFs binary. Publication PDFs use suppressed volatile metadata and three clean pdfTeX passes. Verification compares independently generated outputs byte-for-byte and then performs semantic or numerical checks that are not satisfied by byte identity alone.

1.4 Private and external inputs

The root `prompt.txt` is local-only and ignored. Its historical size/hash record is frozen without publishing its current contents. The `Pre-Universe_opus-fable-main` directory and ZIP are external read-only references and are intentionally not tracked by this repository.

1.5 Tested toolchain and required capabilities

The final Phase 7 public-clone verification used the following toolchain. These are tested versions, not claims that every component requires exactly these minimum versions.

Tool	Verified version	Required for
Python	3.14.5	Generators, independent checkers, tests, notebook build/execution.
Rust compiler	1.91.1	Four application crates and the pinned solver crates.
Cargo	1.91.1	Formatting, linting, tests, release binaries, numerical runs.
Git for Windows	2.51.2	History, tags, recursive submodule checkout, integrity checks.
WolframScript	1.14.0, Windows 64-bit	Exact Wolfram gates and Mathematica notebook generation/execution.
MiKTeX-pdfTeX	4.27 in MiKTeX 26.5	Three-pass deterministic PDF builds.
<code>nbformat</code>	5.10.4	Jupyter notebook parsing and deterministic serialization.
<code>nbclient</code>	0.10.2	Fresh-kernel Jupyter execution.
Jupyter kernel	named <code>python3</code>	Execution of <code>notebooks/dirac_triality.ipynb</code> .

Most Python generators and checkers use only the standard library. The Jupyter execution path additionally needs `nbformat`, `nbclient`, and a registered `python3` kernel. The repository does not track a root `requirements.txt`; developers must verify these packages in the selected interpreter before running Phase 4.

PowerShell gates find `pdflatex.exe` on `PATH` or use `C:\Program Files\MiKTeX\miktex\bin\x64`. Bash gates resolve the Windows executables through `scripts/resolve_wolframscript.sh`. A plain Windows shell does not imply that Visual Studio tools are absent merely because `cl.exe` is not on `PATH`; load a Visual Studio Developer environment before diagnosing a native-toolchain failure.

1.6 Clone and bootstrap

Clone recursively because the solver is a Git submodule:

```
git clone --recurse-submodules https://github.com/once-ere/dirac.git
Set-Location dirac
git -C vendor/sundials_rs rev-parse HEAD
```

The expected solver commit is d1836e6a279d63a90fe2839a0020123245487e76. If an existing clone omitted submodules, run:

```
git submodule update --init --recursive
```

Phases 1 through 7 can validate tracked canonical inputs from a normal public recursive clone. A complete Phase 0 rebuild additionally requires the seven sibling directories named in `config/source-roots.json`; they are historical source corpora and are not vendored into this repository.

1.7 Configuration ownership

`config/source-roots.json` is the explicit Phase 0 list of seven sibling source corpora. Adding or renaming a root changes the frozen audit boundary and requires a complete Phase 0 rebuild and human review.

`config/audit-decisions.json` is curated review data, not generated output. It records per-root or per-artifact execution status, relevance, defects, disposition, and the review document containing the evidence. Machine generation may validate its schema and coverage but must not invent human review conclusions.

1.8 Audit disposition vocabulary

The Phase 0 ledger uses a bounded vocabulary:

Disposition	Developer meaning
historical-evidence	Preserve as lineage/context; do not treat embedded output as a current proof.
independently-reconstruct	Re-derive the relevant result from declared conventions rather than trusting source claims.
reuse-pattern	An engineering pattern may be reused only with project-owned inputs and stronger checks.
generated	The artifact must be traced to its generator and freshly executed or rebuilt.
mixed-see-review	Root-level contents have different dispositions; consult the named file in <code>audit/reviews/</code> .

The structural audit never silently promotes historical output into verified evidence. The authoritative explanation for a disposition is the associated human-review Markdown file.

2 Top-level architecture

Path	Ownership and role
<code>artifacts/</code>	Canonical exact fixtures, numerical histories, summaries, and notebook reports.
<code>audit/</code>	Source manifest, structural audit, review decisions, ledger, and checksums.
<code>config/</code>	Source-root and audit-decision configuration.
<code>dissertation/</code>	Primary and teaching-edition Markdown, LaTeX, and PDFs.
<code>notebooks/</code>	Generated Mathematica and Jupyter notebooks.
<code>provenance/</code>	Phase-specific mathematical, numerical, and publication reports.
<code>refinement/</code>	Focused post-release audits and machine-readable claim evidence.
<code>scripts/</code>	Generators, independent checkers, logged runners, and phase gates.
<code>studies/</code>	Application-owned Rust/CVODE crates.
<code>tests/</code>	Python regression, mutation, canonical-hash, and semantic tests.
<code>vendor/sundials_rs/</code>	Pinned safe-Rust SUNDIALS 7.8.0 solver submodule.
<code>wolfram/</code>	Standalone Wolfram exact and numerical verification sources.

At the Phase 7 release, Git tracks 368 root-repository entries, including 64 under `scripts/`, 20 under `tests/`, 18 under `studies/`, 24 under `provenance/`, 17 under `artifacts/`, 13 under `audit/`, 8 under `refinement/`, 6 under `dissertation/`, 6 under `wolfram/`, and 3 under `notebooks/`. The submodule contents are tracked by the solver repository and are not included in those root-path counts.

2.1 Developer path index

The root and package manifests are:

```
Cargo.toml
studies/triality_transport/Cargo.toml
studies/spinor_cosmology/Cargo.toml
studies/einstein_spinor_44/Cargo.toml
studies/weitzenbock_spinor_44/Cargo.toml
```

Phase 0 configuration/output and primary publication sources are:

```

config/source-roots.json
config/audit-decisions.json
audit/source-manifest.json
audit/audit-ledger.md
dissertation/dirac-triality.md
dissertation/Learn_dirac-triality.md

```

The final refinement evidence and human record are:

```

refinement/phase7-x0-x7/evidence.json
refinement/phase7-x0-x7/convergence.json
refinement/phase7-x0-x7/VERIFICATION.md

```

3 Build and verification graph

The dependency flow is intentionally one-way:

```

frozen source roots
-> source manifest and structural audit
-> exact algebra fixtures
-> generated Rust constants
-> application-owned CVODE studies
-> independent numerical checkers
-> Wolfram/Mathematica/Jupyter publications
-> Markdown -> deterministic LaTeX -> deterministic PDF
-> phase gates -> public fresh-clone verification

```

No publication is the source of an exact or numerical result. Reports consume or cite canonical fixtures and histories; checkers recompute claims from those artifacts and from independently implemented formulas.

3.1 Generator, artifact, checker, and gate ownership

Slice	Generator or implementa- tion	Canonical output	Independent checker	Owning gate
Source audit	build_source_manifest.py, audit_wolfram.wls, build_structural_audit.py	audit/source-manifest.*, wolfram-structure.json, structural-audit.json, ledger, checksums	check_phase0.py and Python audit tests	verify_phase0

Slice	Generator or implementation	Canonical output	Independent checker	Owning gate
Clifford seed	wolfram/ Cl44.wl, verify_cl44.wls	artifact s/exact/ cl44-seed.json	check_cl44_fixture.py	verify_p hase1_seed, verify_phase1
Split octonions	wolfram/ SplitOctonion.wl, verify_split_octonion.wls	artifact s/exact/ split-octonion.json	check_split_octonion_fixture.py	verify_phase1
Triality	wolfram/ Triality44.wl, verify_triality44.wls	artifact s/exact/ triality44.json	check_triality44_fixture.py	verify_phase1
Triality transport	generate_triality_transport_constants.py, Rust crate	history.csv, summary.json, generated Rust	check_triality_transport.py	verify_p hase2_transport
Spinor cosmology	generate_spinor_cosmology_constants.py, Rust crate	background.csv, summary.json, generated Rust	check_spinor_cosmology.py	verify_p hase3_cosmology
Curved spin geometry	build_curved_spin_geometry.py	geometry JSON and Wolfram report	verify_curved_spin_geometry.wls, check_curved_spin_geometry.py	verify_p hase5_curved_spin_gravity
Einstein-spinor	generate_einstein_spinor_constants.py, Rust crate	history.csv, summary.json, generated Rust	check_einstein_spinor_model.py, check_einstein_spinor_44.py	Phase 5 and Phase 7 gates
Weitzenboeck geometry	build_weitzenboeck_spin_geometry.py	geometry JSON and Wolfram report	verify_weitzenboeck_spin_geometry.wls, check_weitzenboeck_spin_geometry.py	verify_p hase6_weitzenboeck_spinor

Slice	Generator or implementation	Canonical output	Independent checker	Owning gate
Weitzenboeck spinor	<code>generate_weitzenboeck_spinor_constants.py</code> , Rust crate	history, summary, generated Rust	<code>check_weitzenboeck_spinor_model.py</code> , <code>check_weitzenboeck_spinor_44.py</code>	Phase 6 gate
Phase 7 claim evidence	refinement builders and exact verifier	<code>evidence.json</code> , <code>converge_nce.json</code>	<code>check_phase7_x0_x7_reports.py</code> and mutation tests	Phase 7 gate
Publications	Markdown and notebook generators	notebooks, TeX, PDFs	notebook, content, and PDF checks	Phase 4 through 7 gates

3.2 Generated Rust contract

Each numerical crate has an application-owned `src/generated.rs` produced from canonical exact or geometry fixtures. The generated source embeds the matrices and input SHA-256 values needed by the application; it is tracked so a clean clone builds without first running a generator. The owning phase gate runs the generator again into canonical and repeat paths and requires the bytes to match. Developers must edit the generator or upstream fixture, never hand-edit `generated.rs`.

The four application manifests depend only on local paths `vendor/sundials_rs/crates/sundials_core` and `vendor/sundials_rs/crates/cvode_rs`. The vendor repository is excluded from the root Cargo workspace because it has its own workspace, verification corpus, and release discipline; root application commands still resolve those two path dependencies normally.

3.3 Notebook generation and execution

Artifact	Generator	Executor/checker	Determinism rule
<code>wolfram/dirac_triality.wls</code>	Project-owned standalone source	WolframScript in Phase 4	Expected check count and zero failures/messages.
<code>notebooks/DiracTriality.nb</code>	<code>build_mathematica_notebook.wls</code>	<code>verify_mathematica_notebook.wls</code> in a fresh kernel	Fixed generated cell inventory, zero messages, expected checks.
<code>notebooks/dirac_triality.ipynb</code>	<code>build_jupyter_notebook.py</code>	<code>run_jupyter_notebook.py</code>	Fixed source structure and deterministic metadata.

Artifact	Generator	Executor/checker	Determinism rule
notebooks/dirac_triality.execute d.ipynb	Fresh execution of source notebook	check_jupyter_notebook.py	Adjacent same-channel streams are coalesced; execution timing metadata is removed.

Notebook code consumes canonical artifact files from the repository root. Historical embedded notebook outputs are never substituted for a fresh-kernel run.

3.4 Logged execution and backup behavior

`run_logged.ps1` and `run_logged.sh` wrap gate commands. A log records UTC start time, repository, exact command, combined output, UTC finish time, and exit code. Logs are local scratch under `logs/` and are not canonical evidence unless a tracked report quotes and independently checks their results.

Before replacing tracked generated files, gates call `backup_files.py`. It copies each file into a UTC-stamped `backups/edits` directory, computes source and backup SHA-256/size, and atomically writes a manifest requiring every copy to match. Backup payloads are ignored; their manifests may remain as local verification evidence.

4 Phase map

4.1 Phase 0: source audit and bootstrap

Phase 0 freezes seven configured source roots with recursive streaming SHA-256, size, type, encoding, and logical-path records. Markdown and Wolfram-family files receive structural indexing and human-review decisions. The audit distinguishes parsed source, hash-only binary artifacts, duplicate lineage, relevance, defects, and disposition. The gate regenerates the manifest, Wolfram structure, structural audit, ledger, and checksums and requires no tracked drift.

Primary entry points:

- `scripts/build_source_manifest.py`;
- `scripts/audit_wolfram.wls`;
- `scripts/build_structural_audit.py`;
- `scripts/check_phase0.py`;
- `scripts/verify_phase0.ps1` and `scripts/verify_phase0.sh`.

4.2 Phase 1: exact algebra and triality

Phase 1 builds a real integral 16-by-16 representation of $\text{Cl}(4,4)$, verifies its full and even algebra ranks, constructs rank-eight half-spin projectors, and establishes irreducibility and inequivalence after restriction to $\text{Spin}(4,4)$. It then constructs split-octonion multiplication and para-product tensors and solves the split-real triality related-triple equations.

Canonical fixtures:

- `artifacts/exact/cl44-seed.json`;
- `artifacts/exact/split-octonion.json`;
- `artifacts/exact/triality44.json`.

The full Clifford module is real, 16-dimensional, and irreducible for $\text{Cl}(4,4)$. Under $\text{Spin}(4,4)$ it decomposes as the direct sum of two inequivalent real eight-dimensional half-spin modules. Code and documentation must not collapse those distinct statements.

4.3 Phase 2: triality transport

The `studies/triality_transport` crate integrates a 24-state system formed by three linked eight-dimensional triality sectors. Generated constants are hash-linked to the exact fixtures. The application uses the pinned CVODE engine without changing vendor code. Independent checks validate sample grids, source hashes, equation residuals, invariant drift, summaries, and byte-identical replay.

4.4 Phase 3: homogeneous spinor cosmology

The `studies/spinor_cosmology` crate integrates an 18-state real homogeneous nonlinear-spinor system and compares it with analytic condensate, density, potential, and Friedmann relations. This phase establishes the numerical pattern later reused and independently rederived in the curved Einstein-spinor applications.

4.5 Phase 4: notebooks and publications

Phase 4 generates and executes standalone WolframScript, Mathematica, and Jupyter artifacts. It builds the primary dissertation and a parallel teaching edition from Markdown to deterministic standalone LaTeX and warning-free PDFs. Notebook gates validate structure, fixed cell inventories, execution results, canonical hashes, and replay. The publication gate requires independent PDF builds to match byte-for-byte.

4.6 Phase 5: curved Levi-Civita Einstein-spinor model

Phase 5 constructs an explicit curved signature $(4,4)$ metric, vielbein, Christoffel symbols, Levi-Civita spin connection, real rank-16 spinor bundle, and invariant split bilinear. Exact Python and

Wolfram paths independently verify the geometry and homogeneous Dirac contraction.

The `studies/einstein_spinor_44` crate solves an 18-state coupled system with no separate scalar field and no cosmological constant. Its selected potential is

$$V(S) = \frac{1}{20}S + \frac{19}{20}S^{1/5}, \quad S = \psi^\top C\psi.$$

The linear term is dust-like at homogeneous background level. The fractional term has equation of state $w=-4/5$ and supplies negative pressure. These are model-internal analogies, not observational identifications.

4.7 Phase 6: Weitzenboeck spin geometry

Phase 6 replaces the Levi-Civita spin connection with a flat inertial Weitzenboeck connection in a declared proper frame. The geometry is metric-compatible, curvature-free, and torsionful. Exact checks verify the contortion decomposition, torsion scalar, TEGR boundary identity, and homogeneous spinor-operator relation. A separate application-owned Rust crate reproduces the reduced background dynamics without depending on the Phase 5 application crate.

4.8 Phase 7: $\{x_0, \dots, x_7\}$ component refinement

Phase 7 adopts only the coordinate names and ordering $\{x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7\}$ from the external bridge reference. It does not import that reference’s metric or connection. The refinement corrects the negative metric entries at indices 5, 6, and 7; separates covariant PDEs from the homogeneous system; expands the exact condensate and all 16 scalar spinor ODEs; and inventories all 8 diagonal and 28 independent off-diagonal homogeneous Einstein equations.

The tracked workspace `refinement/phase7-x0-x7` contains a claim ledger, exact/canonical evidence, convergence evidence, and a public-clone verification record. The final verified tag is `phase7-x0-x7-refinement-green`.

4.9 Phase checkpoint matrix

Counts below describe the named phase checkpoint, not a sum of all prior checks rerun by a later gate.

Phase	Content or verified commit	Tag	Principal verification result
0	763b1734512adf19 6ccf1465a77875a1 a20c21b6	phase0-source-au dit-green	11 tests; 5,991 files; 371,942,252 bytes; 209 reviewed artifacts; 66 message-free Wolfram parses.

Phase	Content or verified commit	Tag	Principal verification result
1	07ed08d46d194f72e4fbea89df6fdeedfa61eafe	phase1-exact-triality-green	Clifford 23 Wolfram + 24 independent; split octonions 17 + 17; triality 25 + 24.
2	1b7ba925c1bb8b830494a70d0dba3e7fd467991b	phase2-triality-transport-green	3 Rust tests, 15 output checks, deterministic replay; 41 samples, 237 steps, 251 RHS calls.
3	e6d149a061a087f55621ed95ce0203738bcae932	phase3-spinor-cosmology-green	3 Rust tests, 17 output checks, deterministic replay; 1,201 samples, 711 steps, 783 RHS calls.
4	ab030d78a60d220f69f30aa7a9d587785e4c6f52	phase4-publication-green	69 Wolfram, 5 Mathematica, 12 Jupyter, 6 PDF checks, and 21 Python tests at the checkpoint.
Learn	b4646d80ca9764b6bb2cd5c2ae9d9298da3c8360	learn-dissertation-green	18 content and 6 PDF checks; complete teaching edition; public-clone verification.
5	f9fef6a47b9cb5767c2f1150e9795798bfe8adef	phase5-curved-spin-gravity-green	Geometry 12 generator + 11 Wolfram + 16 independent; 4 exact model; 24 output; 4 Rust; 33 Python.
6	acf02fe2f4f79fd4d41066aaddde06fb8bdcce97	phase6-weitzenbock-spinor-green	Geometry 15 generator + 15 Wolfram + 20 independent; 13 model, 7 Rust, 34 output, 9 Mathematica, 6 PDF, 38 Python.
7	d0352e66ecff8dcb825555400b77c0e9101da78	phase7-x0-x7-refinement-green	6 matrix, 4 exact model, 7 solver-source, 22 canonical, 24 convergence, 32 semantic, 6 focused, 4 Rust, 42 full Python, and 12 PDF checks.

4.10 Release lineage

The original Phase 4 final release is tagged `final-release-green` at `f18d12fb3a798917edd132d9d57ea10653228b51`. Later tags preserve the Learn, curved-gravity, teleparallel, and component-refinement milestones rather than moving old tags. The scientific baseline summarized here is `d0352e66ecff8dcbb825555400b77c0e9101da78`, tagged `phase7-x0-x7-refinement-green`; later documentation-only commits do not move that scientific tag.

The Phase 7 tag is annotated. Its tag object is `9fd461b38e29e32c2f47351dbb58b9a7f505b20d` and it peels to the final commit. The final recursive public clone passed the complete Phase 7 gate, all 42 Python tests, strict root and submodule `git fsck`, and zero tracked/staged drift.

4.11 Canonical artifact snapshot

Artifact	SHA-256 or invariant
<code>artifacts/exact/cl44-seed.json</code>	0660e436fdcf90ed0f0481c9a828e92659987f5820084ccbf5e1dfc96866cc54
<code>artifacts/exact/split-octonion.json</code>	25044ea194c1e18e40341da1f0098b80bfeb9f4bcb6e7665a32d03f55ac360cabfeb9f4bcb6e7665a32d03f55ac360ca
<code>artifacts/exact/triality44.json</code>	5985f5c0fdd6656c3c9f572ef1a5d06ee21aabe44cc9bf33747bb79b5e181dde5ad82ee604f072f04bed8ef20cdd856bb02ba80e16c3973b10c6c43f972b9c61b8fe5a8a72e48d40d40d6850a6c559e2dd06ccfa9de97982590d35a82044fba6f7ace34949d82d4022a9b5a660733ed018c91b2b931d934aab7793b5585c28f1e45a37c14cc10c7c6b6469ceca7c65e7b9ffc08b3c9dd5bfc518065c590db6b5eab6b001d69c5face7b179af8a27e3cfe254cf80bc740cdcb7a2855fcdf679553b36f201ef43a92c7de3fe2f46457a592e55285e6220d8aa7af6e36a26f9c6b15d084178d01aaed37b84b4bcf8f5246a6941bbe8366ad9c0c3db099eea0f1804f00f31ffa4247fc1e30d8e89df3ea7ddb7c8d9fe795317bd57155c93774cc88ed61cafa59ea0d7693da41527c7b4486ded7cc09ac680b4f20f893d42b62ad03a90539887702ad6bdbe0ee3a5d783094b052609fc0e4d2b761a12b97fa6c858373d6aeea90807a3f064f395b8367502ff3a280050a5c8823c9e0cc0b2eb0b9cf9e89a2326786e2573eafbf1ad1cdc b1528eeb540bfb102cbf5c6988a1ef76

Hashes are contracts enforced by checkers and gates. If a deliberate source change alters one, regenerate and independently verify the downstream chain before updating the expected value.

5 Rust application architecture

The root Cargo workspace contains four application crates:

Crate	State size	Purpose
triality_transport	24	Coupled transport of three triality-linked eight-vectors.
spinor_cosmology	18	Homogeneous nonlinear-spinor background study.
einstein_spinor_44	18	Curved Levi-Civita Einstein-spinor system.
weitzenbock_spinor_44	18	Independent teleparallel/Weitzenboeck counterpart.

Each crate owns its right-hand side, configuration, output schema, and tests. Generated Rust constants are derived from hash-pinned exact fixtures. Each application writes decimal CSV histories and JSON summaries with deterministic formatting. Independent Python checkers parse those outputs and reimplement key formulas rather than calling the Rust right-hand side.

5.1 Common numerical artifact schema

Every study summary identifies its schema version, study name, state/base dimensions where applicable, source fixture hashes, numerical parameters, integration interval and output step, sample count, solver statistics, error measurements, and a **SUCCESS** or **FAILURE** verdict. Histories contain the raw state columns plus derived diagnostics needed for independent recomputation.

Study	State dimension	Samples	CVODE steps	RHS evaluations	Canonical history name
Triality transport	24	41	237	251	artifacts/triality-transport/trajectory.csv
Spinor cosmology	18	1,201	711	783	artifacts/spinor-cosmology/background.csv
Einstein-spinor	18	171	1,372	1,486	artifacts/einstein-spinor-44/history.csv

Study	State dimension	Samples	CVODE steps	RHS evaluations	Canonical history name
Weitzenboeck spinor	18	171	1,372	1,486	artifact s/weitze nbock-sp inor-44/ history.csv

The output checkers require exact headers, expected sample grids, finite values, fixture-hash links, recomputed derived columns, bounded residuals, canonical hashes, and expected solver statistics. Repeat and refined comparisons are enabled by explicit checker arguments rather than inferred from filenames.

5.2 Error semantics

An “error” column is meaningful only with its reference definition. For the Einstein-spinor study, the three roughly $1\text{e-}9$ quantities compare condensate to $\hat{a}^{-7}$, density to its analytic reconstruction, and H^2 to density. They are not ODE residual norms. The five-point output-grid check has a separate maximum component RMS residual of about $4.71\text{e-}4$; the normalized spatial Einstein residual is about $5.70\text{e-}7$. The refined run differs from the canonical state by at most $6.31\text{e-}10$ in the normalized comparison.

6 Exact mathematical core

6.1 Clifford representation

The gamma generators are real integral 16-by-16 matrices satisfying

$$\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab} I_{16}, \quad \eta = \text{diag}(1, 1, 1, 1, -1, -1, -1, -1).$$

Rank computations certify the full matrix algebra and even subalgebra. The chirality projector splits the module into two rank-eight summands. Commutant and intertwiner calculations establish irreducibility and inequivalence over the real numbers.

6.2 Split octonions

The split-octonion fixture records all 64 integral structure constants, conjugation, quadratic norm, ordinary multiplication, and para-product data. Exact tests establish the unit, norm composition, alternativity, and tensor nondegeneracy. The implementation does not assume associativity.

6.3 Triality

The related-triple linear system has dimension 28. Three pairwise inequivalent eight-dimensional modules preserve a nondegenerate trilinear form. A six-element outer action realizes the triality permutations, and an explicit invertible link connects the split-octonion model to the canonical Clifford representation.

7 Curved spin geometry

The homogeneous Phase 5/7 metric uses **x4** as proper evolution coordinate:

$$g_{\mu\nu} = \text{diag}(a^2, a^2, a^2, a^2, -1, -a^2, -a^2, -a^2).$$

A constant-**x4** slice has signature (4,3). The spin derivative convention is

$$D_\mu = \partial_\mu + \frac{1}{8}\omega_{\mu ab}[\gamma^a, \gamma^b],$$

with both ordered tangent indices summed. For a homogeneous spinor, the Levi-Civita contraction is

$$\gamma^\mu D_\mu \psi = \gamma^4 \left(\partial_{x^4} + \frac{7}{2}H \right) \psi.$$

The covariant action, stress tensor, unrestricted field equations, homogeneous component expansion, and exact sign conventions are documented in the Phase 5 and Phase 7 provenance reports.

8 Einstein-spinor numerical method

The production solver uses the pinned safe-Rust SUNDIALS CVODE 7.8.0 port: variable-step, variable-order BDF (orders 1 through 5), default Newton iteration, an internal dense difference-quotient Jacobian, and an 18-by-18 dense direct linear solve. No comparative benchmark establishes this as a globally optimal integrator; it is the selected robust implementation.

At **x4=0**, the canonical state has **a=1**, **H=1**, mathematical spinor components **psi_1=1**, **psi_16=1/2**, and all others zero. The application integrates independently backward to **-0.2** and forward to **1.5**, then joins the ordered samples.

Exact skew-adjointness gives

$$\dot{S} = -7HS, \quad S = a^{-7}.$$

This reduces the background to one quadrature and the rescaled spinor to eight rotations generated by **gamma^4**. The canonical CVODE history is also checked by five-point finite differences for all 18 state equations and by a tighter convergence run.

9 Verification strategy

9.1 Layered independence

Verification is layered to avoid circular confidence:

1. fixture generators check internal invariants;
2. independent Python and Wolfram implementations reconstruct exact claims;
3. Rust unit tests exercise application-owned equations and domains;
4. Python output checkers recompute derived quantities from CSV state columns;
5. deterministic replay compares independent runs byte-for-byte;
6. refined runs test convergence under tighter tolerances and smaller steps;
7. semantic document checkers require formulas, scope statements, and claim boundaries, not only hashes;
8. mutation tests prove that altered PDFs, torsion data, or reintroduced matrix sums are rejected;
9. fresh recursive clones test line endings, submodule pins, paths, and public availability.

9.2 Phase gates

The authoritative PowerShell and Bash gates are:

```
verify_phase0  
verify_phase1  
verify_phase2_transport  
verify_phase3_cosmology  
verify_phase4_publication  
verify_phase5_curved_spin_gravity  
verify_phase6_weitzenbock_spinor  
verify_phase7_x0_x7_reports
```

Every gate uses logged commands and exits nonzero on a failed check. Generated files are backed up before replacement. Backup payloads and local logs are ignored; canonical artifacts and verification code remain tracked.

9.3 Publication checks

Markdown is authoritative for human-facing publications. The shared converter produces standalone LaTeX. Two isolated three-pass builds must have the same bytes. PDF checks validate header, EOF marker, page count, media box, canonical SHA-256, and repeat identity. Logs are scanned for TeX errors, warnings, overfull boxes, underfull boxes, and undefined controls.

10 Developer workflows

10.1 Fast repository status

```
.\scripts\status.ps1
```

```
./scripts/status.sh
```

10.2 Complete Python regression suite

```
python -m unittest discover -s tests -v
```

10.3 Full current verification sequence

```
.\scripts\verify_phase0.ps1
.\scripts\verify_phase1.ps1
.\scripts\verify_phase2_transport.ps1
.\scripts\verify_phase3_cosmology.ps1
.\scripts\verify_learn_dissertation.ps1
.\scripts\verify_phase4_publication.ps1
.\scripts\verify_phase5_curved_spin_gravity.ps1
.\scripts\verify_phase6_weitzenbock_spinor.ps1
.\scripts\verify_phase7_x0_x7_reports.ps1
```

The Bash sequence is explicit rather than implied:

```
bash ./scripts/verify_phase0.sh
bash ./scripts/verify_phase1.sh
bash ./scripts/verify_phase2_transport.sh
bash ./scripts/verify_phase3_cosmology.sh
bash ./scripts/verify_learn_dissertation.sh
bash ./scripts/verify_phase4_publication.sh
bash ./scripts/verify_phase5_curved_spin_gravity.sh
bash ./scripts/verify_phase6_weitzenbock_spinor.sh
bash ./scripts/verify_phase7_x0_x7_reports.sh
```

On Windows Bash/WSL, invoke the phase scripts with **bash** when executable-bit handling is uncertain.

10.4 Rust quality checks

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
```

10.5 Publication regeneration

Use the relevant phase publication gate rather than invoking pdfTeX manually. The gate performs backups, Markdown conversion, isolated builds, warning scans, structure checks, canonical-hash checks, and replay comparisons.

10.6 Common failure modes

Symptom	Likely cause	Correct response
<code>cl</code> , <code>link</code> , or <code>nmake</code> missing in a plain shell	Visual Studio environment not loaded	Locate Visual Studio with <code>vswhere.exe</code> , then run the appropriate <code>vcvarsall.bat</code> before concluding the toolchain is absent.
A canonical text hash changes only on Windows	<code>core.autocrlf</code> changed bytes	Inspect <code>.gitattributes</code> , identify the artifact's writer, and pin the correct <code>eol</code> ; do not simply replace the expected hash.
Two PDF builds differ	Insufficient TeX passes, volatile metadata, path-sensitive input, or toolchain change	Remove build directories, run three passes, scan logs, and compare source/TeX before updating any PDF hash.
Notebook hash changes between executions	Kernel metadata, stream chunking, or environment mismatch	Use <code>run_jupyter_notebook.py</code> ; confirm stream coalescing and the selected Windows Python environment.
Phase 0 reports missing roots	Public clone lacks the seven sibling corpora	Restore the configured directories beside the repository or limit verification to phases whose inputs are tracked.
Evidence JSON differs in a fresh clone	A hashed source has checkout-dependent bytes or an absolute path leaked into output	Compare source records, pin line endings, store repository-relative paths, rebuild, and repeat the public-clone test.
Numerical replay differs	Solver commit/toolchain/input/format changed, or nondeterministic code entered the path	Verify submodule commit and generated constants, compare summaries first, then inspect state histories and RHS checks.
Wolfram assertions pass but messages occur	Kernel messages are not automatically fatal	Use the project verifier that captures <code>\$MessageList</code> and requires both the expected success count and zero messages.
Gate leaves new <code>backups/</code> <code>edits/*</code> directories	Expected pre-edit backup behavior	Keep payloads local; do not stage them as deliverables.

10.7 Changing the solver pin

Changing `vendor/sundials_rs` is a high-impact operation:

1. review the new solver commit independently and preserve its license data;
2. update the submodule pointer without editing vendor files from the root project;
3. run vendor tests in the submodule workspace;
4. run formatting, clippy, unit tests, canonical replay, and refined runs for all four root applications;
5. regenerate every affected numerical summary and report;
6. verify PowerShell and Bash gates in a recursive public clone;
7. record both old and new commit IDs and all changed numerical measurements.

The root workspace excludes the vendor workspace intentionally; never add the entire solver tree as root Cargo members merely to make one command broader.

10.8 Adding a new source root or scientific artifact

For a new Phase 0 source root, update `config/source-roots.json`, regenerate the manifest, classify every new Markdown/Wolfram-family artifact, add human review evidence, and rerun Phase 0 from frozen inputs. For a new canonical scientific artifact, define its schema, generator, independent checker, deterministic serialization, expected check count, provenance report, and owning phase gate before treating it as release material.

11 Canonical artifacts and provenance

The repository separates source, generated artifacts, and reports:

- exact truth is stored in machine-readable JSON fixtures;
- numerical truth is stored in canonical CSV/JSON runs;
- notebook output is generated from verified artifacts;
- provenance Markdown explains conventions, commands, limitations, and hashes;
- LaTeX and PDF are deterministic derivatives of Markdown;
- `PROGRESS.md` records phase checkpoints and release history;
- `refinement/phase7-x0-x7/VERIFICATION.md` records the latest independent public-clone audit.

Developers changing a canonical artifact must update its generator, independent checker, frozen hashes, downstream publication, and phase gate in the same change. Updating only an expected hash is not a valid verification strategy.

11.1 Current publication snapshot

Artifact	Bytes	SHA-256	Structural result
Primary dissertation Markdown	28,697	44b76c2d872598ad 1b038a8d4ad1293b 18885ab228297c92 1579daa86eeb1381	Authoritative source.
Primary dissertation LaTeX	30,979	12df60627b5f11b0 9ccbadd9ca38fd2 ce7efcfccd769f69 fd8280b78a6cec38	Deterministic derivative.
Primary dissertation PDF	396,932	8531af531c91fbf3 66fbc30759da63e 3881dcc241931698 a9f6ee3a6ae0b69e	19 letter-sized pages.
Learn dissertation Markdown	102,388	ab6c653f6d0f2355 411d61c9357d3c70 9ced8b9ac7016786 1e893a9029052676	Authoritative teaching source.
Learn dissertation LaTeX	118,691	533eeae0619d3b5 90d74fa03a4a81aa a76163fce289d659 a1007d1b10b0e03e	Deterministic derivative.
Learn dissertation PDF	745,001	7c683b51445a3b49 64b244ea3e232bc0 ba3f745b5427308b 59a80d1532e19ff9	74 letter-sized pages.
Mathematica notebook	7,036	5c80d2d9367610db 9835f4c570966dba 1751cad77ae30f42 dc0987bc917505e2	Generated Phase 6-complete notebook.
Jupyter source notebook	6,574	5813727a077a9afc c881e86928e7e483 76056d48ce31409f b5c28e00d0844411	Deterministic source notebook.
Executed Jupyter notebook	14,756	2d998c596ac81be0 60562ed0d9a324c9 37b5069521320d95 372f6a2dbce6b169	Normalized fresh-kernel result.
Standalone WolframScript	3,481	260558b921fd3d45 c28a0c757aeb5dd9 b23c0862394c12d5 d874417506728dc4	Project-owned executable report.
Phase 7 components PDF	310,195	f64e3f8420c18c38 86e201c64ee41d8e 0564d7222038f5a8 b83f45a46b7113f4	8 letter-sized pages.
Phase 7 numerics PDF	325,859	8501d99270928203 9b5a69d1de045660 018505f1048f09fe cfedc40795484af9	8 letter-sized pages.

The older Phase 4 dissertation hashes remain valid for the historical Phase 4 tag. The table above describes the Phase 6-complete files on current `main`. Never compare a current artifact against a historical tag's expected hash without first identifying the intended checkpoint.

11.2 Authority hierarchy

When two files disagree, resolve ownership in this order:

1. declared mathematical conventions and project-owned generators;
2. canonical exact fixtures or numerical state artifacts;
3. independent executable checkers and machine-readable evidence;
4. current provenance Markdown;
5. deterministic LaTeX/PDF derivatives;
6. historical notebooks, reviews, or archived outputs.

This hierarchy does not make generators infallible. A generator and its output can agree on the same bug; that is why independent derivations and mutation tests are required.

12 Release discipline

A release checkpoint requires:

1. all focused and full tests pass;
2. the relevant PowerShell and Bash gates pass;
3. generated artifacts have no tracked drift;
4. the solver submodule is clean and at the pinned commit;
5. `git fsck -full -strict` passes in root and submodule;
6. local `HEAD`, `origin/main`, and live GitHub `main` agree;
7. a fresh recursive public clone reproduces canonical hashes;
8. private prompts and external references remain absent from the remote;
9. the release tag peels to the independently verified commit.

The latest verified tag at the time of this summary is `phase7-x0-x7-refinement-green`.

13 Known boundaries and non-claims

The repository proves and reproduces mathematical and numerical statements inside declared conventions. It does not establish:

- that a split-signature $(4,4)$ manifold is the observed spacetime;
- a compactification or signature-change mechanism;
- quantum consistency of a commuting classical spinor;
- perturbative, ghost, or causal stability of the cosmological model;
- dark-matter clustering or a particle candidate;
- an observational dark-energy fit;
- uniqueness or global optimality of the selected potential or integrator;
- that historical notebooks or embedded outputs constitute proof.

Those are future research questions, not implied results.

14 Change-impact guide

Change	Required follow-up
Clifford generator convention	Regenerate all exact fixtures, generated constants, applications, notebooks, and reports; rerun Phases 1–7.
Split-octonion basis/product	Regenerate split-octonion and triality fixtures; rerun transport and all publications.
Solver submodule commit	Reaudit vendor bytes/license/platform behavior; rebuild every Rust study and canonical output.
Einstein-spinor potential	Redo action/stress derivation, analytic laws, Rust RHS, numerical runs, semantic evidence, and dark-sector interpretation.
Metric or frame convention	Rebuild geometry in Python and Wolfram, regenerate spin connections, and rerun Phase 5 onward.
Markdown converter or TeX toolchain	Rebuild every publication twice, update canonical PDF hashes, and fresh-clone verify.
Line-ending rule	Rebuild every hash-sensitive text artifact in a Windows fresh clone.
Notebook generator	Re-execute from a fresh kernel and update structure/result/canonical checks together.

15 Developer checklist

Before editing:

1. identify the owning generator and independent checker;
2. confirm whether the target is source, canonical output, or derived publication;
3. back up tracked generated files with `scripts/backup_files.py`;
4. keep external references and local prompts outside Git.

Before pushing:

1. run focused tests immediately after the first change;
2. run the owning phase gate;
3. run the complete Python suite for shared checkers or publication code;
4. inspect `git diff -check` and tracked status;
5. push only intended files.

After pushing:

1. compare local, tracking, and live remote refs;
2. clone recursively into a new directory;
3. rerun the owning gate and complete tests;
4. require strict root/submodule Git integrity and zero tracked drift;
5. record exact hashes, counts, commit, tag, and scientific boundaries.

16 Documentation map

Start with these files, in order:

1. `README.md` for orientation and current commands;
2. this Developer Summary for repository architecture and maintenance;
3. `PROGRESS.md` for phase and release history;
4. `provenance/CL44_SEED.md` and `provenance/SPLIT_OCTONION_TRIALITY.md` for exact algebra;

5. `provenance/TRIALITY_TRANSPORT.md` and `provenance/SPINOR_COSMOLOGY.md` for early numerical studies;
6. `provenance/CURVED_SPIN_BUNDLE.md` and `provenance/EINSTEIN_SPINOR_44.md` for curved spin gravity;
7. `provenance/WEITZENBOCK_SPINOR_44.md` for teleparallel geometry;
8. the two `EINSTEIN_SPINOR_44*_X0_X7.md` reports for refined component and numerical detail;
9. `refinement/phase7-x0-x7/CLAIM_LEDGER.md` and `VERIFICATION.md` for the latest claim audit.

17 Rebuilding this summary

This Markdown file is authoritative. Its LaTeX and PDF derivatives are built and checked by the dedicated Developer Summary gate:

```
.\scripts\verify_developer_summary.ps1
```

```
bash ./scripts/verify_developer_summary.sh
```

The gate regenerates the LaTeX twice, builds the PDF twice with three passes, checks semantic coverage and canonical hashes, scans all TeX logs, and requires byte identity. Five focused Developer Summary tests cover canonical artifacts, semantic completeness, title type, phase omission, and corrupted-hash rejection. With those tests added, the repository-wide Python suite contains 47 tests at this documentation checkpoint.